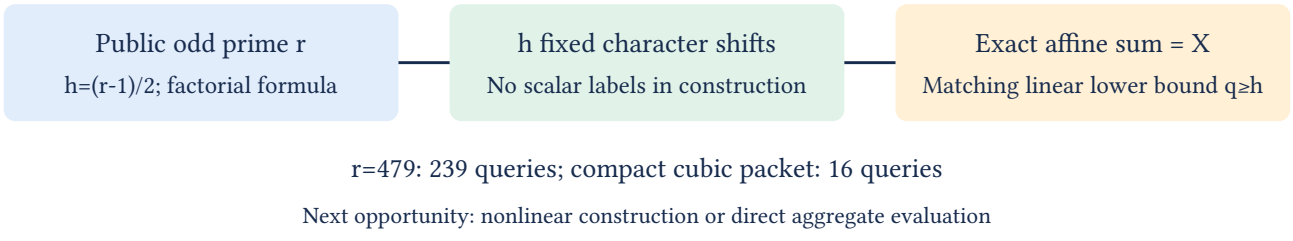
Universal linear character constructor

2026-11-05 · T-1609

A closed factorial formula now constructs an exact scalar decoder from the odd prime r alone. A matching lower bound establishes its optimal shift count within fixed affine-linear combinations of shifted quadratic characters.



Identity and sharp scope

Let h=(r-1)/2 and n=h-1. Euler's criterion identifies the character with X^h. The n-th finite difference of X^h is h! times (X+n/2). Thus the coefficient at shift a is (-1)^(n-a)/(h a! (n-a)!), with constant -n/2. There are exactly h nonzero character coefficients.

If fewer than h distinct shifts represented X by an affine-linear combination, comparing the highest polynomial coefficients would give an invertible Vandermonde system with zero right-hand side. All character coefficients would vanish, a contradiction. This bound permits a constant term and public affine input scalings; it does not cover nonlinear or adaptive decoders.

Implementation and exact checks

The native constructor uses one verified factorial inverse and backward inverse-factorial recurrence. Construction takes O(h+log r) field products and O(h) memory. An explicit max_terms limit is checked before allocation. Primality remains caller-certified.

All 41 native tests pass with overflow checks: 2197 input identities across 31 prime fields; 30 independent polynomial-coefficient checks; error/limit controls; and all eight saved public-point source integrations with 239-sign windows. Sixteen source/data hashes verify before and after build. Four compiler warnings remain recorded.

What this enables

The constructor removes label-dependent interpolation for this linear family. At r=479 it requests 239 signs, versus 16 for the compact cubic packet. These counts are derived; no runtime gain or cloud experiment is asserted. The next structural task is compact nonlinear coefficient construction, or direct evaluation of the aggregate without h source queries. The original 4563-case cohort remains separate from these checks.

Evidence and prior work

See DERIVATION.md, lib.rs, source-lock.json, receipt.json and raw logs. Standard finite-difference background: Illinois course notes and NIST DLMF 26.8. The report gives a direct finite-field proof; no novelty priority claim is made.